Zadanie rekrutacyjne — Unity Developer

Kontekst projektu

Pracujesz nad systemem edukacyjnych aplikacji VR. Jedna z aplikacji pozwala studentowi budować układ elementów na stole montażowym — student pobiera elementy z biblioteki, umieszcza je na stole, łączy ze sobą, a system waliduje poprawność układu.

Twoim zadaniem jest zbudowanie **prototypu systemu biblioteki elementów, ich umieszczania na stole, łączenia i walidacji układu**.

Nie wymagamy grafiki — elementy mogą być prymitywami (sześciany, walce). Nie wymagamy VR — interakcja myszą i klawiaturą.

Wymagania podstawowe (Mid + Senior)

1. Biblioteka elementów

Zaprojektuj i zaimplementuj system biblioteki elementów dostępnych do umieszczenia na stole. Każdy element musi mieć zdefiniowane: nazwę, kategorię, rozmiar, wizualizację (kolor lub kształt prymitywu) oraz porty połączeń (patrz punkt 4).

Biblioteka powinna zawierać **minimum 6 elementów w 3 kategoriach** (po 2 w każdej). Oczekujemy, że samodzielnie zaproponujesz strukturę danych i architekturę tego systemu.

2. Panel UI biblioteki

- Panel wyświetlający elementy z biblioteki jako listę z nazwą i kategorią.
- Filtrowanie po kategorii (dropdown lub zakładki).
- Kliknięcie elementu tworzy jego instancję na scenie.

3. Umieszczanie i zarządzanie elementami na stole

- Scena zawiera „stół" (płaszczyzna).
- Po wybraniu elementu z biblioteki pojawia się on przed kamerą.
- Element można przeciągnąć myszą i upuścić na stole.
- Element snapuje się do powierzchni stołu (nie unosi się w powietrzu).
- Po kliknięciu prawym przyciskiem myszy na element pojawia się **menu kontekstowe** z opcjami:
    - **Edycja** — otwiera panel z informacjami o elemencie (nazwa, kategoria, lista portów i ich stan połączenia).
    - **Usuń** — usuwa element ze stołu (wraz ze wszystkimi jego połączeniami).

4. System portów i połączeń

Każdy element ma zdefiniowane porty bezpośrednio w swoich danych konfiguracyjnych (nie w kodzie). Port ma:

- **Typ**: Input lub Output.
- **Nazwę** (np. "Wyjście ciśnienia", "Wejście oleju").
- **Listę kompatybilnych elementów** — lista kategorii lub konkretnych elementów które mogą być podłączone do tego portu.

Porty są wizualizowane jako małe sfery na krawędziach elementu. Element może mieć **wiele portów Input** (np. element mieszający przyjmuje dwa surowce) i jeden lub więcej portów Output.

Połączenie tworzy się przez kliknięcie portu Output jednego elementu, a następnie portu Input drugiego. Połączenie jest wizualizowane linią (LineRenderer) między portami. Połączenie można usunąć.

Zdefiniowane elementy i ich połączenia

Przygotuj układ symulujący prostą linię produkcyjną:

Element	Kategoria	Porty Output	Porty Input	Kompatybilne połączenia
Zbiornik surowca A	Źródło	1× wyjście	brak	Output → Mieszalnik, Filtr
Zbiornik surowca B	Źródło	1× wyjście	brak	Output → Mieszalnik
Filtr wstępny	Przetwarzanie	1× wyjście	1× wejście	Input ← Zbiornik A; Output → Mieszalnik
Mieszalnik	Przetwarzanie	1× wyjście	2× wejście	Input ← Zbiornik A, Zbiornik B, Filtr; Output → Podgrzewacz
Podgrzewacz	Przetwarzanie	1× wyjście	1× wejście	Input ← Mieszalnik; Output → Pakowarka
Pakowarka	Końcowy	brak	1× wejście	Input ← Podgrzewacz

Przykład poprawnego układu (jeden z możliwych):

Zbiornik A → Filtr → Mieszalnik → Podgrzewacz → Pakowarka, gdzie Zbiornik B podłączony jest do drugiego wejścia Mieszalnika.

5. Walidacja układu

Przycisk „Sprawdź układ" uruchamia walidację. System sprawdza:

- **Kompatybilność połączeń** — czy każde istniejące połączenie łączy elementy które są na swojej liście kompatybilnych połączeń. Połączenie niekompatybilne = błąd z opisem (np. „Zbiornik B nie może być podłączony bezpośrednio do Podgrzewacza").
- **Kompletność wyjść** — czy wszystkie porty Output są podłączone. Niepodłączony port Output = ostrzeżenie ze wskazaniem elementu i portu.

Wynik walidacji wyświetlany w panelu UI:

- Lista błędów i ostrzeżeń z opisem.
- Elementy z błędami podświetlone na czerwono.
- Elementy z ostrzeżeniami podświetlone na żółto.
- Poprawny układ: wszystkie elementy podświetlone na zielono + komunikat sukcesu.

6. Zapis i wczytanie układu

- Przycisk „Zapisz" — serializuje aktualny stan układu (pozycje elementów, typy, połączenia) do pliku JSON.
- Przycisk „Wczytaj" — deserializuje JSON i odtwarza układ na stole.
- JSON powinien być czytelny (sformatowany, zrozumiałe nazwy pól).

Wymagania rozszerzone (Senior)

7. Architektura modułowa

- Użyj **Assembly Definitions** — minimum 3: Core (biblioteka, porty, walidacja), UI (panele), App (scena, kontroler gry).
- UI i App referencjonują Core, ale Core nie referencjonuje żadnego z nich.
- Komunikacja między modułami przez **interfejsy** zdefiniowane w Core lub przez **eventy** (delegates/UnityEvents).

8. Wzorzec walidacji

- Interfejs ILayoutValidator z metodą ValidationResult Validate(LayoutState state).
- Minimum **dwie implementacje**: walidator kompatybilności połączeń i walidator kompletności wyjść (z wymagań podstawowych), ale wydzielone jako osobne klasy spełniające wspólny interfejs.

- Walidatory rejestrowane w kolekcji i uruchamiane sekwencyjnie — dodanie nowego walidatora nie wymaga zmiany istniejącego kodu.
- ValidationResult zawiera: listę błędów z typem (błąd/ostrzeżenie), opisem i referencją do problematycznego elementu.

9. Konfigurowalność przez dane

- Stwórz **wzorzec układu** jako ScriptableObject (LayoutTemplateSO): lista wymaganych elementów z typami i wymaganymi połączeniami.
- Dodaj trzeci walidator: TemplateComplianceValidator — porównuje układ studenta ze wzorcem i raportuje odchylenia (brakujące elementy, nadmiarowe elementy, błędne połączenia).
- Wzorzec edytowalny w inspektorze Unity bez zmiany kodu.

10. Tryby oceny

- Enum EvaluationMode: Learning, Test.
- **Learning**: walidacja automatyczna po każdej zmianie (dodanie, połączenie, usunięcie). Błędne elementy natychmiast podświetlone.
- **Test**: walidacja tylko po kliknięciu „Sprawdź układ". Brak podpowiedzi w trakcie pracy.
- Przełącznik trybu w UI (przed startem lub w ustawieniach).

Forma dostarczenia

- Projekt Unity (2022.3 LTS lub nowszy, URP).
- Repozytorium Git z historią commitów (chcemy zobaczyć proces pracy, nie jeden wielki commit).
- Plik README.md z:
  - Instrukcją uruchomienia.
  - Opisem podjętych decyzji architektonicznych.
  - Listą rzeczy które byś zmienił/dodał gdybyś miał więcej czasu.
- **Czas: 3–5 dni** od otrzymania zadania.

Uwagi

- Nie oceniamy grafiki — prymitywy są OK.
- Nie wymagamy VR — mysz i klawiatura wystarczą.

- Kandydaci na stanowisko **Mid** realizują wymagania z sekcji „Wymagania podstawowe".
- Kandydaci na stanowisko **Senior** realizują wymagania z obu sekcji.
- Cenimy kod który jest **łatwy do rozszerzenia** — wyobraź sobie że za miesiąc ktoś inny będzie dodawał nowy typ walidatora i nową kategorię elementów.
- Jeśli czegoś nie zdążysz — opisz w README co i jak byś zrobił. Wolimy mniejszy zakres zrobiony dobrze niż cały zakres zrobiony byle jak.